



NIB Appointment - System Design Document

Contents

NIB Appointment - System Design Document	1
1. Architectural Overview	2
1.1 Architecture Style	2
1.2 High-Level Components	2
2. Data Flow Overview	3
2.1 Authentication & Authorization Data Flow	4
2.1.1 User Login	4
2.1.2 Middleware Enforcement	4
2.1.3 Role-Based Permissions	5
3. Patient Portal Data Flow	5
3.1 Appointment Booking (Standalone Web)	5
3 Mini-App Booking	5
4. Professional Portal Data Flows	5
5. Data Storage & Relationships	6
6. API & Server Actions	6
7. Security	6
8. External Integrations	6
Conclusion	7

Introduction

This document describes the system design for the Appointment Application. It provides an overview of the system architecture, major components, data flow, and integrations. The goal is to give stakeholders and developers a clear understanding of how the system works without going into low-level implementation details.

System Overview

The Appointment Application is a web-based platform that allows patients to book appointments with doctors across different hospitals.

The system supports multiple user roles:

- ✓ Patient
- ✓ Doctor
- ✓ Hospital Admin
- ✓ Super Admin.

The application handles appointment scheduling, user management, payment processing, and reporting.

1. Architectural Overview

The system follows a client-server architecture implemented as a single web application with role-based access control.

1.1 Architecture Style

- ✓ Monolithic web application with modular components
- ✓ Role-based portals within the same application
- ✓ Server-side rendering and API handling within a unified backend

1.2 High-Level Components

- ✓ Web Client (Browser)
- ✓ Application Server
- ✓ Authentication Module
- ✓ Appointment Management Module
- ✓ Payment Integration Module

✓ Database

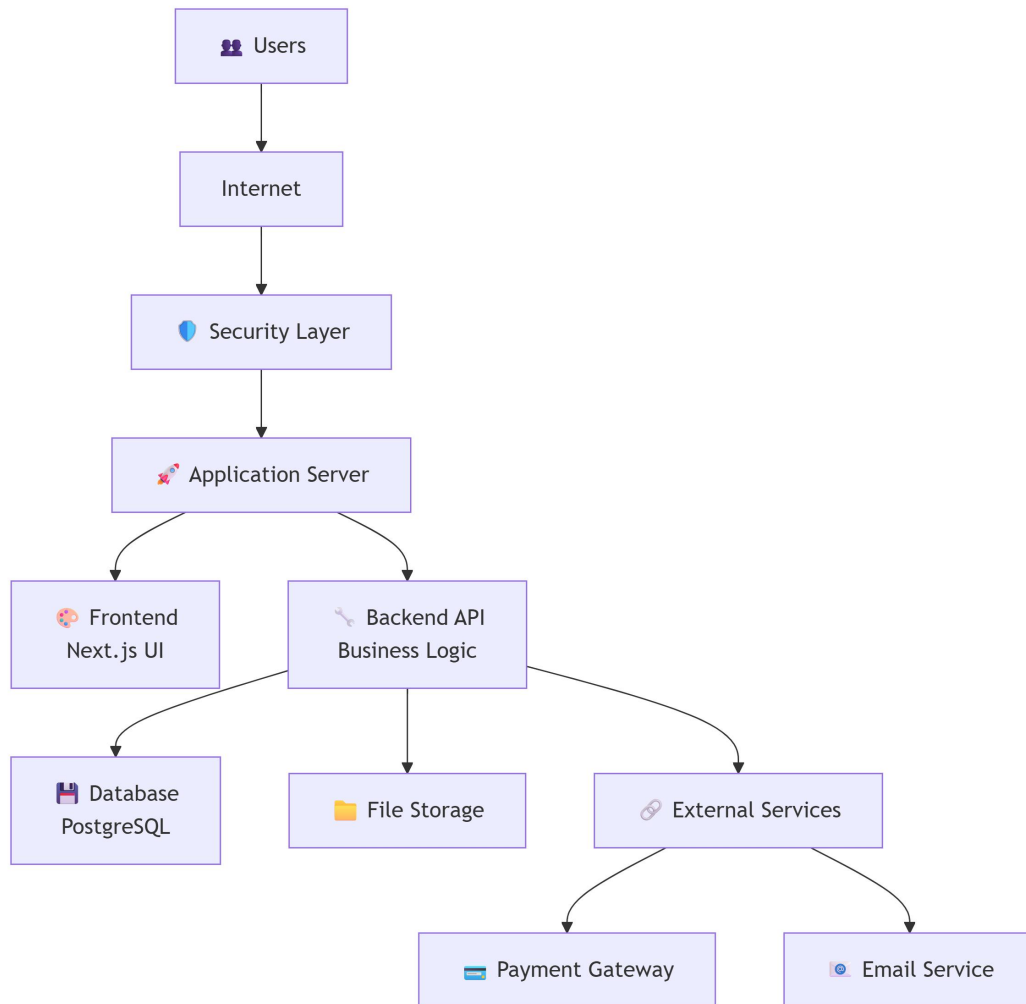


Image 1: System Architecture

2. Data Flow Overview

Key data flows include session-based patient identification, appointment booking, payment processing, and reporting.

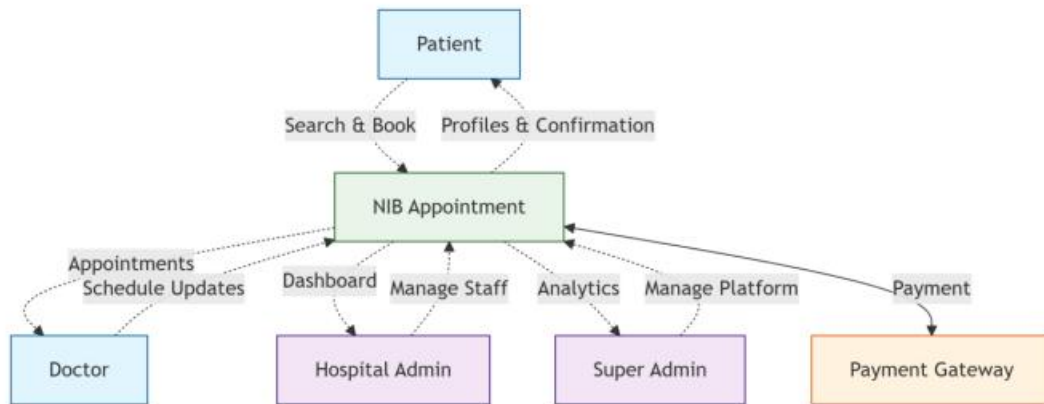


Image 2:DFD diagram

2.1 Authentication & Authorization Data Flow

2.1.1 User Login

Users submit credentials:

- ✓ Email & password for doctors, hospital staff, and admins
- ✓ Basic patient details for appointment booking (no authentication required)

Authentication flow:

- ✓ Credentials validated in auth.ts
- ✓ Passwords verified using bcrypt
- ✓ Rate limiting enforced (5 attempts per email, 20 per IP)
- ✓ NextAuth generates JWT with role, permissions, token version
- ✓ Session stored in HTTP-only cookies

2.1.2 Middleware Enforcement

- ✓ JWT validated on every request
- ✓ Role-based access control
- ✓ Forced password change checks

- ✓ Mini-app session validation
- ✓ Hospital staff permission filtering

2.1.3 Role-Based Permissions

- ✓ Permissions resolved via User, Role, RolePermission, Permission models
- ✓ Permission keys embedded in JWT
- ✓ UI and routes gated by permissions

3. Patient Portal Data Flow

3.1 Appointment Booking

- ✓ Browse doctors and hospitals
- ✓ Filter by specialty, city, availability
- ✓ Submit booking form
- ✓ Appointment created with confirmed status

Mini-App Booking

- ✓ Appointment created as pendingpayment
- ✓ Payment via external gateway
- ✓ Callback confirms payment
- ✓ Appointment status updated to confirmed

4. Professional Portal Data Flows

Doctor Portal:

- ✓ Appointment dashboard
- ✓ Schedule management

Hospital Admin Portal:

- ✓ Doctor management
- ✓ Appointment and queue management

Super Admin Portal:

- ✓ Hospital CRUD
- ✓ Maker-checker approval flow

5. Data Storage & Relationships

Core Models:

- ✓ SuperAdmin, Hospital, Doctor, Patient, Appointment, DoctorSchedule, User, Role, Permission

6. API & Server Actions

- ✓ Auth, payment callback, appointment status polling
- ✓ Prisma ORM with Zod validation

7. Security

- ✓ JWT authentication
- ✓ Role-based authorization
- ✓ Secure file uploads
- ✓ Token expiration and versioning

8. External Integrations

- ✓ Payment gateway
- ✓ Super App JWT validation
- ✓ Email services

Conclusion

This System Design Document presents a clear and structured overview of the Appointment Application, including its architecture, core components, user roles, data flow, and external integrations. It is intended to serve as a shared reference for stakeholders, developers, and integration partners to ensure a common understanding of how the system operates.